CHAPTER 15

Existential Types and Dynamic Dispatch

In the last three chapters the *caller* always chose the type. When you call `mconcat::<Sum>`, or let the compiler infer the type for you, you pick one instance of a polymorphic function and use it. The function promises to work for every type you might pick; that is what $\forall$ means.

Often the roles are reversed. A function that builds an iterator chain wants to return it without naming its type, which may be impossible to write down anyway, since closures have anonymous types. A configuration file decides at run time which of several drivers a program uses, and the program wants to keep them all in one `Vec`. A library wants to hide how its data is represented, so that it can change the representation later. In each case the *callee* (or the implementer) chooses the type, and the caller is told only what it can do with it. This is the other quantifier, $\exists$: "there is some type, and here is what you may do with its values".

Existential types are the theory behind Rust's `impl Trait` in return position and `dyn Trait`, behind abstract data types and module systems, and behind the vtables that make dynamic dispatch work. They also explain an error message that puzzles many Rust programmers, the one about traits that are not "dyn compatible".

In this chapter you will learn:

- the existential type $\exists \alpha.\, \tau$, its pack and unpack rules, and why a side condition on the unpack rule is what makes a type abstract;
- why abstract data types have existential type (Mitchell and Plotkin), and how this relates to modules and information hiding;
- the three places a quantifier can go in Rust (generics, `impl Trait` in return position, `dyn Trait`), who chooses the type in each, and when;
- what a `dyn Trait` value is in memory, measured with `size_of`: a fat pointer to the value and its vtable;
- why some traits cannot be used with `dyn` (E0038), explained from the typing rules;
- `dyn Any` and downcasting: an existential that can be opened, and what that costs;
- how to encode $\exists$ with $\forall$, in Rust and in Lean, and why a Lean structure with a type field lives in `Type 1`.

15.1 Hiding a type

Who chooses

A universally quantified type $\forall\alpha.\,\tau$ is a promise made by the *implementer* of a function to its *caller*: "whatever type you choose for α, I will work". The caller chooses, the implementer must cope with any choice. An existentially quantified type $\exists\alpha.\,\tau$ reverses the roles: "I have chosen some type α, and I give you a τ that mentions it; you must cope with whatever I chose". The implementer chooses, and the client must work without knowing the choice.

Chapter 10 met the logical side of this: a proof of $\exists x.\,P(x)$ consists of a witness a and a proof of $P(a)$. In Lean it has two incarnations, the proposition `∃ x, P x` and dependent pairs such as `{ x // P x }`, which are data. An existential *type* is the same idea one level up: the witness is a type.

Pack and unpack

Let us extend the types of System F (Chapter 12) with existentials:

$$\tau ::= \ldots \mid \exists\alpha.\,\tau$$

The introduction form **packs** a type together with a value; the elimination form **unpacks** a package and gives the client a name for the hidden type and a name for the value:

$$\frac{\Gamma \vdash e : \tau[\alpha := \sigma]}{\Gamma \vdash \mathit{pack}\ \langle \sigma, e\rangle\ \mathit{as}\ \exists\alpha.\,\tau : \exists\alpha.\,\tau}\ \textsf{T-Pack}$$

$$\frac{\Gamma \vdash e_1 : \exists\alpha.\,\tau \qquad \Gamma, \alpha, x : \tau \vdash e_2 : \rho \qquad \alpha \notin \mathrm{ftv}(\rho)}{\Gamma \vdash \mathit{unpack}\ \langle \alpha, x\rangle = e_1\ \mathit{in}\ e_2 : \rho}\ \textsf{T-Unpack}$$

Rule T-Pack says: if e has the type τ with the *concrete* type σ in place of α, then you may package σ and e together and forget that the type was σ. The resulting type, $\exists\alpha.\,\tau$, no longer mentions σ; many different packages, with different hidden types, have this same type.

Rule T-Unpack is the interesting one. To use a package e_1, you write a client e_2 that must type-check with a *fresh* type variable α in the context and a variable x of type τ. Since α is a variable, the client cannot assume anything about it: it is not `u32`, not `String`, not anything in particular. The client may only use x through whatever operations τ provides. The third premise, $\alpha \notin \mathrm{ftv}(\rho)$ (the type variable α is not among the *free type variables* of the result type ρ), forbids the hidden type from escaping: the client's result must make sense without α, so it cannot be, say, the value x itself. That side condition is what makes the type *abstract*.

The computation rule opens the package and substitutes both of its parts:

$$\mathit{unpack}\ \langle \alpha, x\rangle = (\mathit{pack}\ \langle \sigma, v\rangle\ \mathit{as}\ \exists\alpha.\,\tau)\ \mathit{in}\ e_2 \to e_2[\alpha := \sigma][x := v]$$

The client was checked for an arbitrary α, so it is correct for σ in particular. Notice the asymmetry: *running* the program reveals σ, but *type-checking* the client never does.

If you have read Chapter 4, these rules will look familiar. Erase the terms, and T-Pack and T-Unpack are the introduction and elimination rules for $\exists$ in natural deduction:

$$\frac{\Gamma \vdash P(a)}{\Gamma \vdash \exists x.\,P(x)}\ \exists\,\mathsf{I} \qquad\qquad \frac{\Gamma \vdash \exists x.\,P(x) \qquad \Gamma, P(x) \vdash Q \qquad x \text{ not free in } \Gamma \text{ or } Q}{\Gamma \vdash Q}\ \exists\,\mathsf{E}$$

The elimination rule has the same shape as the one for $\vee$: to use $\exists x.\, P(x)$, show that Q follows from $P(x)$ for an arbitrary x, about which you know nothing else, and Q must not mention x. Logicians call the side condition the *eigenvariable condition*. It is the same restriction as $\alpha \notin \mathrm{ftv}(\rho)$, read in logic instead of programs.

Abstract types have existential type

Here is the existential type of a counter: some state type S, with an initial state, a way to step and a way to read the count.

$$\mathit{Counter} = \exists S.\, \{\, \mathit{init} : S,\ \mathit{step} : S \to S,\ \mathit{get} : S \to \mathbb{N} \,\}$$

One implementation packs $S := \mathbb{N}$ with 0, $n \mapsto n + 1$ and the identity. Another packs $S := \mathrm{List}\ \mathbf{1}$ (a list of units, one per step) with the empty list, "add one element" and "length". Both packages have type $\mathit{Counter}$. A client unpacks one, steps a few times and reads the count, and it cannot tell which representation it got: by rule T-Unpack, it was checked without knowing S.

John Mitchell and Gordon Plotkin made this observation in 1988, in a paper whose title states it: "Abstract types have existential type". An **abstract data type** (a type whose representation is hidden behind a set of operations) *is* an existential package, and the guarantees of data abstraction (clients cannot depend on the representation, so it can be changed without breaking them) follow from the typing rules. We will prove such a guarantee for our two counters in Lean later in the chapter.

You use this idea every day in a weaker form. A Rust module that exports a struct with private fields, and functions that operate on it, is an abstract type: code outside the module cannot see the fields. The ML family of languages goes further, with *signatures* that declare a type without revealing its definition, so that a module can be sealed to hide its representation. What existential types add is that the package is a *value*: you can choose an implementation at run time, store packages in a list, and pass them to functions.

15.2 Three places for a quantifier in Rust

All the Rust code of this section is in `code/rust/examples/ch15_existentials.rs`. The interface is a trait, and there are three implementations with three different state types:

```rust
/// The interface: a counter can take a step and report its value.
trait Counter {
    fn step(&mut self);
    fn get(&self) -> u32;
}

/// Counts by one.
struct Simple(u32);

impl Counter for Simple {
    fn step(&mut self) {
        self.0 += 1;
    }
    fn get(&self) -> u32 {
        self.0
    }
}
```

```
/// Counts by one and keeps a log: a different representation with the same
/// observable behavior.
struct Logged {
    count: u32,
    log: Vec<String>,
}

impl Counter for Logged {
    fn step(&mut self) {
        self.count += 1;
        self.log.push(format!("step {}", self.count));
    }
    fn get(&self) -> u32 {
        self.count
    }
}

/// Counts by a fixed stride: different behavior, too.
struct Stride {
    value: u32,
    by: u32,
}

impl Counter for Stride {
    fn step(&mut self) {
        self.value += self.by;
    }
    fn get(&self) -> u32 {
        self.value
    }
}
```

Rust lets you write code against `Counter` in three ways, and the difference between them is exactly *where the quantifier goes* and *who chooses the type*.

The caller chooses: generics

```
/// ∀C. Counter C ⇒ C → u32 → u32. The *caller* chooses `C`.
fn run_generic<C: Counter>(mut counter: C, steps: u32) -> u32 {
    for _ in 0..steps {
        counter.step();
    }
    counter.get()
}

/// The same quantifier, written with `impl Trait` in argument position.
fn run_impl(mut counter: impl Counter, steps: u32) -> u32 {
    for _ in 0..steps {
        counter.step();
    }
    counter.get()
}
```

`run_generic` has type $\forall C.\ \mathit{Counter}\ C \Rightarrow C \to \mathrm{u32} \to \mathrm{u32}$ (Chapter 14). The caller chooses `C`, and the compiler produces one copy of the function for each choice. The second function shows that **`impl Trait` in argument position is universal**: `fn run_impl(counter: impl Counter, ...)` is another way of writing `run_generic`, with an anonymous type parameter. This is a common source of confusion, since the same syntax in return position means the opposite.

```
println!(
    "generic:    {} {}",
    run_generic(Simple(0), 3),
```

```
    run_impl(Stride { value: 0, by: 10 }, 3)
);
```

```
generic:    3 30
```

The callee chooses, statically: `impl Trait` in return position

```
/// ∃C. Counter C ∧ C. The *callee* chooses the type (here `Logged`); the
/// caller only learns that it implements `Counter`.
fn make_counter() -> impl Counter {
    Logged {
        count: 0,
        log: Vec::new(),
    }
}
```

The signature says: "this function returns a value of *some* type that implements `Counter`". The function's body chooses the type (here `Logged`), and the caller gets a package of type $\exists C.\ \mathit{Counter}\ C \wedge C$: a value of an unknown type, usable only through the trait.

```
let counter = make_counter();
println!("impl Trait: {}", run_generic(counter, 3));
```

```
impl Trait: 3
```

The caller may not rely on the hidden type, even though the compiler knows it perfectly well (`code/rust/compile_fail/ch15_opaque_type.rs`):

```
fn make_counter() -> impl Counter {
    Simple(0)
}

fn peek() -> u32 {
    let counter: Simple = make_counter();
    counter.0
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch15_opaque_type.rs:27:27
   |
22 | fn make_counter() -> impl Counter {
   |                      ------------ the found opaque type
...
27 |     let counter: Simple = make_counter();
   |                  ------   ^^^^^^^^^^^^^^ expected `Simple`, found opaque type
   |                  |
   |                  expected due to this
   |
   = note:   expected struct `Simple`
           found opaque type `impl Counter`
```

Rust calls the type of the result an **opaque type**: `impl Counter` is a type distinct from every other type, which the caller can use only through `Counter`. This is rule T-Unpack at work. The client `peek` would need $\alpha = \texttt{Simple}$, and T-Unpack checks the client for an arbitrary α.

Return-position `impl Trait` is a restricted existential, in two ways. First, the witness is chosen once, *per function* (and per instantiation of the function's type parameters), not per call: all the return statements

must produce the same type, so a function cannot decide at run time between two representations (`code/rust/compile_fail/ch15_impl_two_types.rs`):

```rust
fn make_counter(fast: bool) -> impl Counter {
    if fast {
        Stride { value: 0, by: 10 }
    } else {
        Simple(0)
    }
}
```

```
error[E0308]: `if` and `else` have incompatible types
  --> compile_fail/ch15_impl_two_types.rs:39:9
   |
36 | /     if fast {
37 | |         Stride { value: 0, by: 10 }
   | |         -------------------------- expected because of this
38 | |     } else {
39 | |         Simple(0)
   | |         ^^^^^^^^^ expected `Stride`, found `Simple`
40 | |     }
   | |_____- `if` and `else` have incompatible types
   |
help: you could change the return type to be a boxed trait object
   |
35 - fn make_counter(fast: bool) -> impl Counter {
35 + fn make_counter(fast: bool) -> Box<dyn Counter> {
   |
help: if you change the return type to expect trait objects, box the returned expressions
   |
37 ~         Box::new(Stride { value: 0, by: 10 })
38 |     } else {
39 ~         Box::new(Simple(0))
   |
```

The compiler's suggestion, a boxed trait object, is the third placement, which we look at next. Second, the hiding is for the *programmer*, not the compiler. The compiler knows the hidden type, monomorphizes the caller for it and calls `Logged::step` directly, with no run-time cost. (There is one documented leak: auto traits such as `Send` and `Sync` "see through" an opaque type, so a caller can rely on `make_counter()` being `Send` if `Logged` is, even though the signature does not say so.)

This is the form of existential you use most, often without noticing: every function that returns an iterator chain or a closure as `impl Iterator<...>` or `impl Fn(...)` returns a package whose hidden type is too complicated, or too anonymous, to write down.

The value carries its choice: `dyn Trait`

```rust
/// The witness type is chosen at run time, so it can differ from call to
/// call. The value is packed together with its vtable behind a pointer.
fn make_counter_dyn(kind: &str) -> Box<dyn Counter> {
    match kind {
        "simple" => Box::new(Simple(0)),
        "logged" => Box::new(Logged {
            count: 0,
            log: Vec::new(),
        }),
        _ => Box::new(Stride { value: 0, by: 10 }),
    }
}

/// Unpacking: the client works with any hidden type, through the interface.
fn run_dyn(counter: &mut dyn Counter, steps: u32) -> u32 {
```

```
    for _ in 0..steps {
        counter.step();
    }
    counter.get()
}
```

`Box<dyn Counter>` is an existential package in the full sense. The function chooses the hidden type at run time, differently on each call; each value carries, next to the data, the dictionary (vtable) for its own type; and a client such as `run_dyn` works for every package, through the interface. Since each value carries its own witness, values with different hidden types can live in one collection:

```
// A heterogeneous collection: three different hidden types in one Vec.
let mut counters: Vec<Box<dyn Counter>> = ["simple", "logged", "stride"]
    .into_iter()
    .map(make_counter_dyn)
    .collect();
let results: Vec<u32> = counters
    .iter_mut()
    .map(|c| run_dyn(c.as_mut(), 3))
    .collect();
println!("dyn Trait:  {results:?}");
```

```
dyn Trait:  [3, 3, 30]
```

In the notation of this chapter, `Vec<Box<dyn Counter>>` is $\mathrm{Vec}\ (\exists C.\ \mathit{Counter}\ C \wedge C)$: the quantifier is *inside* the vector, once per element. Compare `Vec<impl Counter>` as a return type, $\exists C.\ \mathit{Counter}\ C \wedge \mathrm{Vec}\ C$, with the quantifier outside and one hidden type shared by all the elements.

A module with an abstract type

The trait `Counter` describes *objects*: each value comes with its own state and operations. Mitchell and Plotkin's abstract data type is a slightly different thing: one hidden type, and a package of operations that create and manipulate *any number* of values of that type. Rust can express this too, with an associated type as the hidden type:

```
/// An abstract data type in the style of Mitchell and Plotkin:
/// ∃S. { init: S, step: S → S, get: S → u32 }. The hidden type is the
/// associated type `State`; a value of a type implementing `CounterModule`
/// is the package of operations.
trait CounterModule {
    type State;
    fn init(&self) -> Self::State;
    fn step(&self, state: Self::State) -> Self::State;
    fn get(&self, state: &Self::State) -> u32;
    /// A binary operation: fine here, because every state of this module has
    /// the same hidden type.
    fn merge(&self, a: Self::State, b: Self::State) -> Self::State;
}

struct UnaryCounters;

impl CounterModule for UnaryCounters {
    /// One `()` per step: a deliberately different representation.
    type State = Vec<()>;
    fn init(&self) -> Vec<()> {
        Vec::new()
    }
    fn step(&self, mut state: Vec<()>) -> Vec<()> {
        state.push(());
        state
```

```
    }
    fn get(&self, state: &Vec<()>) -> u32 {
        state.len() as u32
    }
    fn merge(&self, mut a: Vec<()>, b: Vec<()>) -> Vec<()> {
        a.extend(b);
        a
    }
}

/// The package is returned as `impl CounterModule`: callers cannot learn
/// that the states are vectors.
fn counter_module() -> impl CounterModule {
    UnaryCounters
}

/// A client of the module. It manipulates states of type `M::State` without
/// knowing what they are.
fn two_runs<M: CounterModule>(module: &M, first: u32, second: u32) -> u32 {
    let mut a = module.init();
    for _ in 0..first {
        a = module.step(a);
    }
    let mut b = module.init();
    for _ in 0..second {
        b = module.step(b);
    }
    module.get(&module.merge(a, b))
}
```

`counter_module()` returns a package, $\exists S.\ \{init : S, \ldots\}$, in the shape of the formula at the start of the chapter. The client `two_runs` creates *two* states, manipulates them, and merges them, all without knowing that a state is a `Vec<()>`:

```
let module = counter_module();
println!("module:     {}", two_runs(&module, 2, 3));
```

```
module:     5
```

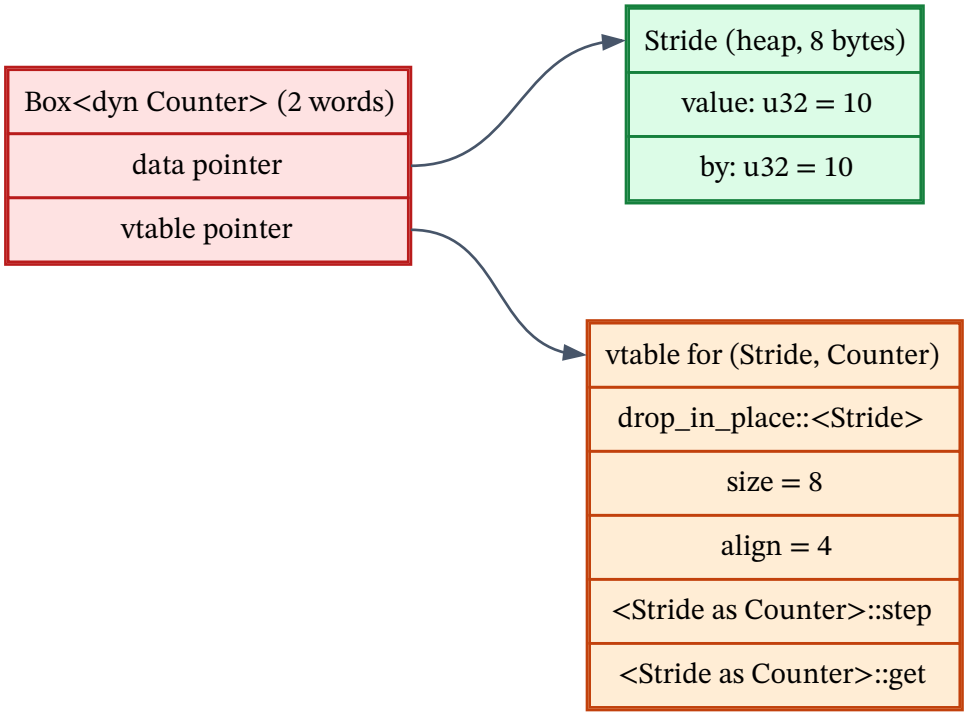
Notice `merge`, a **binary operation** on the hidden type. It is fine here, because both states come from the same package and therefore have the same hidden type. With objects, as we will see in the section on dyn compatibility, binary operations are a problem: two `Box<dyn Counter>` values may hide two different types.

Summary of the three placements

Rust	Quantifier	Who chooses	When the type is known	Dispatch
`fn f<C: Counter>(c: C)`	$\forall$	caller	compile time	static
`fn f(c: impl Counter)`	$\forall$	caller	compile time	static
`fn f() -> impl Counter`	$\exists$, one per function	callee	compile time, hidden	static
`Box<dyn Counter>`	$\exists$, one per value	whoever builds it	run time	vtable

15.3 Trait objects in memory

A value of type `dyn Counter` is a package $\langle \sigma, v \rangle$ whose type σ has been erased: nothing at run time says "this is a `Stride`". What survives of σ is the information the client needs to use v through the interface, and that is a dictionary in the sense of Chapter 14: the vtable for `Stride` as a `Counter`. A pointer to a trait object is therefore two pointers, one to the value and one to the vtable. Rust calls such a pointer a **fat pointer**.



A boxed trait object of type dyn Counter in memory: a data pointer to the value of the hidden type, and a vtable pointer to the dictionary for (Stride, Counter), which also records the hidden type's size and alignment.

The program `code/rust/examples/ch15_fat_pointers.rs` measures pointers with `size_of`, in machine words, so that the output is the same on 32-bit and 64-bit machines:

```
/// The size of `T` in machine words (8 bytes on a 64-bit machine, 4 on a
/// 32-bit one), so that the output does not depend on the platform.
fn words<T>() -> usize {
    size_of::<T>() / size_of::<usize>()
}

/// Print one row of the table.
fn row<T>(name: &str) {
    println!("{name:<26}{}", words::<T>());
}
```

```
println!("{:<26}words", "pointer type");
row::<&u32>("&u32");
row::<&[u32; 4]>("&[u32; 4]");
row::<Box<Simple>>("Box<Simple>");
row::<fn(u32) -> u32>("fn(u32) -> u32");
```

```
row::<&[u32]>("&[u32]");
row::<&str>("&str");
row::<&dyn Counter>("&dyn Counter");
row::<&mut dyn Counter>("&mut dyn Counter");
row::<Box<dyn Counter>>("Box<dyn Counter>");
row::<Rc<dyn Counter>>("Rc<dyn Counter>");
row::<Option<Box<dyn Counter>>>("Option<Box<dyn Counter>>");
row::<&dyn Fn(u32) -> u32>("&dyn Fn(u32) -> u32");
```

```
pointer type              words
&u32                      1
&[u32; 4]                 1
Box<Simple>               1
fn(u32) -> u32            1
&[u32]                    2
&str                      2
&dyn Counter              2
&mut dyn Counter          2
Box<dyn Counter>          2
Rc<dyn Counter>           2
Option<Box<dyn Counter>>  2
&dyn Fn(u32) -> u32       2
```

The first four are thin pointers: a reference to a sized type, a `Box` of one, and a function pointer are one word each. Every pointer to a `dyn` type is two words, whatever the pointer (`&`, `&mut`, `Box`, `Rc`): the second word is the vtable pointer. `Option<Box<dyn Counter>>` is still two words, because a `Box` is never null and `None` can use the null value (the niche optimization of Chapter 11). And `&dyn Fn(u32) -> u32` is two words too: a closure behind a `dyn` pointer is a trait object like any other, which is how Chapter 14's `MonoidDict` stored its functions.

Slices are fat as well: `&[u32]` and `&str` carry a length as their second word. A slice type `[u32]` is, in the same spirit, an existential over a *number*, "some n, and an array `[u32; n]`", and the hidden n travels in the pointer:

```
// A slice reference is a fat pointer too: the hidden quantity is a length.
let readings = [215u32, 218, 221, 219];
let first_two: &[u32] = &readings[..2];
let all: &[u32] = &readings;
println!(
    "slices of length {} and {}: both {} words",
    first_two.len(),
    all.len(),
    words::<&[u32]>()
);
```

```
slices of length 2 and 4: both 2 words
```

An existential over a value rather than a type is a dependent pair, a Σ-type; Chapter 30 returns to it.

The vtable

In the current implementation of Rust, a vtable starts with three entries that every trait object needs (the function that drops the hidden value, and its size and alignment) followed by one entry per method; the exact layout is not guaranteed. The size and alignment are needed because the *type* is gone: to free a `Box<dyn Counter>`, the program must tell the allocator how big the value is, and it can only find out at run time. `size_of_val` and `align_of_val` do exactly that, reading the vtable:

```
// Three packages with three different hidden types.
let mut counters: Vec<Box<dyn Counter>> = vec![
    Box::new(Small(0)),
    Box::new(Simple(0)),
    Box::new(Stride { value: 0, by: 10 }),
];
// The size and alignment of each hidden type are not known statically;
// `size_of_val` and `align_of_val` read them from the vtable.
for counter in counters.iter_mut() {
    counter.step();
    let hidden: &dyn Counter = counter.as_ref();
    println!(
        "value {:>2}: hidden type of size {} and alignment {}",
        hidden.get(),
        size_of_val(hidden),
        align_of_val(hidden),
    );
}
```

```
value  1: hidden type of size 2 and alignment 2
value  1: hidden type of size 4 and alignment 4
value 10: hidden type of size 8 and alignment 4
```

The three packages hide types of sizes 2, 4 and 8 bytes, and the program finds this out from three different vtables. A consequence is that `dyn Counter` has no size known at compile time: it is a **dynamically sized type**, which can exist only behind a pointer. The compiler rejects a parameter of type `dyn Counter` (`code/rust/compile_fail/ch15_unsized.rs`):

```
fn value_of(counter: dyn Counter) -> u32 {
    counter.get()
}
```

```
error[E0277]: the size for values of type `(dyn Counter + 'static)` cannot be known at
compilation time
  --> compile_fail/ch15_unsized.rs:10:22
   |
10 | fn value_of(counter: dyn Counter) -> u32 {
   |                      ^^^^^^^^^^^ doesn't have a size known at compile-time
   |
   = help: the trait `Sized` is not implemented for `(dyn Counter + 'static)`
help: you can use `impl Trait` as the argument type
   |
10 - fn value_of(counter: dyn Counter) -> u32 {
10 + fn value_of(counter: impl Counter) -> u32 {
   |
help: function arguments must have a statically known size, borrowed types always have a
known size
   |
10 | fn value_of(counter: &dyn Counter) -> u32 {
   |                      +
```

Its two suggestions are the two quantifiers. `impl Counter` makes the function universal, so that each caller's type is known at compile time; `&dyn Counter` keeps the existential but puts it behind a pointer that carries the vtable.

Packing and unpacking in Rust

In Rust, rule T-Pack is a **coercion**: where a `Box<Stride>` is used as a `Box<dyn Counter>`, the compiler converts it by adding the vtable pointer for `(Stride, Counter)`, a constant it generates once for each pair of type and trait. That conversion is called an *unsizing coercion*, and it happens implicitly at the

points where the expected type is known, as in the arms of `make_counter_dyn`. Rule T-Unpack is a method call: `counter.step()` on a `&mut dyn Counter` loads the `step` entry from the vtable and calls it with the data pointer. Inside `Stride::step`, `Self` is `Stride` again; the client never learns it.

The hidden type may contain references, and the existential must say how long they live. The full form of the type is `dyn Counter + 'a`: some type that implements `Counter` *and outlives* `'a`. When you omit the bound, Rust fills in a default: for `Box<dyn Counter>` it is `'static` (the box may live forever, so the hidden type must not borrow anything temporary), and for `&'a dyn Counter` it is `'a`. That is the `'static` that Chapter 14's dictionaries had to require of their type parameters. Chapter 26 is about lifetimes.

Dynamic dispatch has costs that `impl Trait` avoids: an indirect call that the compiler usually cannot inline, a heap allocation for each `Box`, and fat pointers. What it buys is the run-time choice of type, heterogeneous collections, and a single copy of the client's code instead of one per type.

15.4 Dyn compatibility

Not every trait can be used as `dyn Trait`. Here is one that cannot (`code/rust/compile_fail/ch15_dyn_incompatible.rs`):

```
trait Mergeable {
    fn total(&self) -> u32;
    fn merge(&self, other: &Self) -> Self;
    fn accept<V: Visitor>(&self, visitor: &mut V);
}

fn store(items: &mut Vec<Box<dyn Mergeable>>) {}
```

```
error[E0038]: the trait `Mergeable` is not dyn compatible
  --> compile_fail/ch15_dyn_incompatible.rs:17:30
   |
17 | fn store(items: &mut Vec<Box<dyn Mergeable>>) {}
   |                              ^^^^^^^^^^^^^ `Mergeable` is not dyn compatible
   |
note: for a trait to be dyn compatible it needs to allow building a vtable
      for more information, visit <https://doc.rust-lang.org/reference/items/traits.html#
dyn-compatibility>
  --> compile_fail/ch15_dyn_incompatible.rs:13:28
   |
11 | trait Mergeable {
   |       --------- this trait is not dyn compatible...
12 |     fn total(&self) -> u32;
13 |     fn merge(&self, other: &Self) -> Self;
   |                            ^^^^^     ^^^^ ...because method `merge` references the
`Self` type in its return type
   |                            |
   |                            ...because method `merge` references the `Self` type in this
parameter
14 |     fn accept<V: Visitor>(&self, visitor: &mut V);
   |        ^^^^^^ ...because method `accept` has generic type parameters
   = help: consider moving `merge` to another trait
   = help: consider moving `accept` to another trait
```

The rules for which traits are **dyn compatible** (older documentation says "object safe") are listed in the Rust Reference and in `rustc --explain E0038`. They can look arbitrary, but each one follows from the typing rules of existentials and from the representation we have just seen. Think of a call through

`dyn Mergeable` as an unpack: the receiver is a package $\langle \alpha, x \rangle$ with α unknown, and the call must type-check for that unknown α using only one vtable entry.

A method that returns `Self`. `merge` returns a `Self`, that is, a value of type α. After unpacking, the result type of the call would mention α, violating $\alpha \notin \mathrm{ftv}(\rho)$; and at the machine level the caller would not know how much space to reserve for the result. (A method returning `&Self` has the same problem in a milder form.)

A method that takes another `Self`. `merge` also takes `other: &Self`, a second value of the hidden type. But a client holding two `&dyn Mergeable` values holds two packages, which may hide two *different* types, a `Batch` and a `Single`; there is no way to call `merge` on them. This is the classic **binary method** problem of object-oriented type systems, and it is why the module-style existential (one package, many values of the same hidden type) can have binary operations and the object-style one cannot.

A generic method. `accept<V: Visitor>` is polymorphic: its type is $\forall V.\ \mathit{Visitor}\ V \Rightarrow \ldots$. Rust implements polymorphism by monomorphization, so the vtable would need one entry for each type `V` with which `accept` will ever be called, and the vtable is fixed when the package is created, long before anyone decides which `V` to use. A polymorphic function inside an existential package is a *rank-2* type (Chapter 12), and Rust has no single machine-code entry that implements one.

A method without a receiver. A function like `fn new() -> Self` has no `self`, so a call has no package to unpack and no vtable to read it from. (Chapter 14's `Monoid` fails here, with `empty`, and again with `combine`, which takes a second `Self`. That is no accident: a monoid is about combining values of the *same* type, which is the module-style kind of existential.)

The cure, in each case, is either to remove the method from the vtable or to change it so that one vtable entry serves every use. A bound `where Self: Sized` removes a method from `dyn` use (a `dyn` type is never `Sized`), while keeping it available on concrete types. And a generic method can be replaced by one that takes a trait object, so that the method receives the visitor's dictionary at run time instead of being monomorphized for it: dictionary passing, from Chapter 14, replacing monomorphization. The repaired trait is in `code/rust/examples/ch15_dyn_compat.rs`:

```
trait Mergeable {
    fn total(&self) -> u32;

    /// Mentions `Self`: excluded from the vtable, so it can be called on
    /// concrete types only.
    fn merge(&self, other: &Self) -> Self
    where
        Self: Sized;

    /// Takes a trait object instead of a type parameter: one vtable entry
    /// that works for every visitor.
    fn accept(&self, visitor: &mut dyn Visitor);
}
```

```
// `merge` works on concrete types, where both arguments have the same type.
let merged = Batch(vec![3, 1]).merge(&Batch(vec![4]));
println!("merged batch total: {}", merged.total());

// Through `dyn Mergeable`, `total` and `accept` are available...
let items: Vec<Box<dyn Mergeable>> = vec![
```

```
    Box::new(merged),
    Box::new(Single(9)),
    Box::new(Batch(vec![2, 6])),
];
let totals: Vec<u32> = items.iter().map(|item| item.total()).collect();
println!("totals: {totals:?}");

// ...and `accept` works with any visitor, chosen at run time too.
let mut all = Collect(Vec::new());
let mut largest = Largest(0);
for item in &items {
    item.accept(&mut all);
    item.accept(&mut largest);
}
println!("visited: {:?}, largest: {}", all.0, largest.0);
```

```
merged batch total: 8
totals: [8, 9, 8]
visited: [3, 1, 4, 9, 2, 6], largest: 9
```

`merge` is still there, for code that knows the concrete type: merging two batches gives a batch with total 8. Through `dyn Mergeable`, `total` and `accept` work on a heterogeneous collection, and `accept` works with every visitor, each passed as a `&mut dyn Visitor`.

15.5 `Any`: an existential you can open

The standard library has a trait object that breaks the rules on purpose: `dyn Any`. Every type that holds no short-lived references (every `T: 'static`) implements `Any`, and the trait's one method returns a `TypeId`, a unique identifier of the hidden type. With it, the methods `downcast_ref`, `downcast_mut` and `Box::downcast` ask a package "is your hidden type `T`?" and, if the answer is yes, return the value at type `T`. The examples are in `code/rust/examples/ch15_any.rs`.

```
/// A `&dyn Any` hides its type, but `downcast_ref` asks "is it a `T`?" and,
/// if so, returns the value at type `T`.
fn describe(value: &dyn Any) -> String {
    if let Some(n) = value.downcast_ref::<i32>() {
        format!("an i32: {n}")
    } else if let Some(s) = value.downcast_ref::<String>() {
        format!("a String of length {}", s.len())
    } else if let Some(b) = value.downcast_ref::<bool>() {
        format!("a bool: {b}")
    } else {
        "something else".to_string()
    }
}
```

```
let packages: Vec<Box<dyn Any>> = vec![
    Box::new(42i32),
    Box::new(String::from("fridge")),
    Box::new(1.5f64),
];
for p in &packages {
    println!("{}", describe(p.as_ref()));
}
```

```
an i32: 42
a String of length 6
something else
```

In terms of rule T-Unpack, `downcast_ref` is a client that does what the side condition was meant to prevent: it learns the hidden type and lets it escape. The price is that abstraction no longer holds. A client with `Any` can tell our two counter representations apart, which no client using only the interface could do.

The same leak reaches generic functions. A function of type $\forall\alpha.\,\alpha \to \alpha$ cannot do anything with its argument but return it, and Chapter 18 will make this precise: the only total function of that type that cannot inspect types is the identity. Rust gives generic functions a way to inspect types:

```
/// The signature looks like ∀T. T → T, whose only total, parametric
/// inhabitant is the identity. The bound `T: Any` is the warning sign: the
/// body may look at the type, and it does.
fn not_identity<T: Any>(mut value: T) -> T {
    if let Some(n) = (&mut value as &mut dyn Any).downcast_mut::<i32>() {
        *n += 1;
    }
    value
}
```

```
println!(
    "not_identity: {} {:?} {}",
    not_identity(41i32),
    not_identity("unchanged"),
    not_identity(41i64)
);
```

```
not_identity: 42 "unchanged" 41
```

`not_identity` returns its argument unchanged for a `&str` and an `i64`, and adds one to an `i32`. Rust is honest about this, in a way: the bound `T: Any` appears in the signature and warns the reader that the function may look at the type. Not every leak is announced, though. `std::mem::size_of::<T>()` and `std::any::type_name::<T>()` need no bound at all, and this chapter uses both: the `words` helper of `ch15_fat_pointers.rs` is a generic function whose result depends on the type, and the `TypeName` client in the section on encodings prints a type that was supposed to be hidden. Chapter 18 calls these features **parametricity leaks** and studies what they cost.

Downcasting is still useful. A map with one value per type (used, for example, to attach typed extensions to a request) is a classic application:

```
/// A map with at most one value per *type*, as used for request extensions
/// and plugin registries. The key is the hidden type's `TypeId`.
#[derive(Default)]
struct Extensions {
    values: BTreeMap<TypeId, Box<dyn Any>>,
}

impl Extensions {
    fn insert<T: Any>(&mut self, value: T) {
        self.values.insert(TypeId::of::<T>(), Box::new(value));
    }

    fn get<T: Any>(&self) -> Option<&T> {
        self.values.get(&TypeId::of::<T>())?.downcast_ref::<T>()
    }
}

struct RequestId(u64);
```

```
struct Operator(String);
```

```
let mut ext = Extensions::default();
ext.insert(RequestId(7));
ext.insert(Operator("ada".to_string()));
let id: Option<&RequestId> = ext.get::<RequestId>();
let operator: Option<&Operator> = ext.get::<Operator>();
println!("request id: {:?}", id.map(|r| r.0));
println!("operator:   {:?}", operator.map(|o| o.0.as_str()));
println!("a u8:       {:?}", ext.get::<u8>());
```

```
request id: Some(7)
operator:   Some("ada")
a u8:       None
```

And `Box<dyn Error>`, the standard way to return "some error", is an existential whose clients may downcast to the error types they know how to handle:

```
/// `Box<dyn Error>` hides the type of the error; a caller that cares can
/// downcast to the concrete error types it knows how to handle.
fn parse_reading(text: &str) -> Result<i64, Box<dyn Error>> {
    let value: i64 = text.trim().parse()?;
    if value < -2730 {
        return Err("below absolute zero".into());
    }
    Ok(value)
}

fn classify(result: Result<i64, Box<dyn Error>>) -> String {
    match result {
        Ok(v) => format!("ok: {v}"),
        Err(e) => match e.downcast_ref::<ParseIntError>() {
            Some(parse) => format!("not a number ({parse})"),
            None => format!("invalid reading ({e})"),
        },
    }
}
```

```
for text in ["215", "21.5", "-3000"] {
    println!("{text:>6}: {}", classify(parse_reading(text)));
}
```

```
  215: ok: 215
 21.5: not a number (invalid digit found in string)
-3000: invalid reading (below absolute zero)
```

The `'static` requirement of `Any` has a reason: `TypeId` identifies types after lifetimes have been erased, so it could not tell `&'a str` from `&'static str`, and a downcast between them would be unsound.

15.6 Existentials in Lean

Lean has no `exists` type former of the System F kind, because it does not need one: a dependent pair whose first component is a *type* is an existential package. The code is in `code/lean/Book/Ch15/Existential.lean`.

Packages as dependent pairs

```
/-- info: (α : Type) × (α → String) : Type 1 -/
#guard_msgs in
#check ((α : Type) × (α → String))

/-- info: (α : Type) × α × (α → String) : Type 1 -/
#guard_msgs in
#check ((α : Type) × α × (α → String))
```

The type `(α : Type) × (α → String)` is the Lean spelling of $\exists \alpha.\, \alpha \to \texttt{String}$: a pair of a type and a function on it. It is a useless package, though: its client receives a function from α and no α to apply it to. Add a value, and you get something useful, Lean's counterpart of `Box<dyn Display>`:

```
/-- A value of some hidden type, packed with a way to display it: the
counterpart of Rust's `Box<dyn Display>`. -/
abbrev Showable := (α : Type) × α × (α → String)

/-- Packing: choose the hidden type, give a value and an implementation. -/
def readings : List Showable :=
  [⟨Nat, 42, fun n => s!"{n} ticks"⟩,
   ⟨String, "fridge", fun s => s!"sensor {s}"⟩,
   ⟨Bool, true, fun b => if b then "door open" else "door closed"⟩]

/-- Unpacking: the client sees a type `α`, a value of type `α` and a function
on `α`, and can only combine them. -/
def Showable.show (p : Showable) : String :=
  match p with
  | ⟨_, x, f⟩ => f x

/-- info: ["42 ticks", "sensor fridge", "door open"] -/
#guard_msgs in
#eval readings.map Showable.show
```

Each element of `readings` packs a different type with a value and a function on that type. `Showable.show` unpacks with a pattern match: it receives some type `α` (the underscore), a value `x : α` and a function `f : α → String`, and can only apply `f` to `x`.

Look at the type in the `#check` outputs: **`Type 1`**, not `Type`. This is the universe bump that Chapter 12 explained in general. A package contains a type, an element of the universe `Type`, and `Type` itself lives in `Type 1`; so a structure with a field of type `Type` must live in `Type 1` too. Lean's universes are *predicative*: a type that quantifies over, or contains, all the types of universe u lives in a higher universe, because allowing `Type : Type` would make the logic inconsistent (a result known as Girard's paradox).

A counter abstract data type

Here is the counter of the start of the chapter, as a Lean structure with a type field:

```
/-- The operations of a counter whose states have type `σ`. -/
structure CounterOps (σ : Type) where
  init : σ
  step : σ → σ
  get : σ → Nat

/-- An abstract data type: ∃σ. CounterOps σ. A value of this structure is
a package of a type and an implementation. -/
structure Counter where
  State : Type
  ops : CounterOps State
```

```
/-- info: Ch15.Counter : Type 1 -/
#guard_msgs in
#check Counter

/-- info: CounterOps Nat : Type -/
#guard_msgs in
#check CounterOps Nat
```

`CounterOps Nat` is an ordinary type, in `Type`; `Counter`, which contains the type `State` as a field, is in `Type 1`. The three implementations are packages with two different state types:

```
/-- Counting with a natural number. -/
def natOps : CounterOps Nat := { init := 0, step := (· + 1), get := id }

/-- Counting with a list of `()`, one per step: a different representation. -/
def unaryOps : CounterOps (List Unit) :=
  { init := [], step := (() :: ·), get := List.length }

/-- Counting by ten: a different behavior. -/
def tenOps : CounterOps Nat := { init := 0, step := (· + 10), get := id }

/-- Three packages: each hides its state type behind the `Counter` interface. -/
def byOne : Counter := ⟨Nat, natOps⟩
def unary : Counter := ⟨List Unit, unaryOps⟩
def byTen : Counter := ⟨Nat, tenOps⟩
```

Pack and unpack are ordinary functions, and the computation rule holds by definition:

```
/-- Rule T-Pack: a type and an implementation for that type make a package. -/
def Counter.pack (σ : Type) (ops : CounterOps σ) : Counter := ⟨σ, ops⟩

/-- Rule T-Unpack: the client `k` must work for every state type `σ`, and its
result type `ρ` is fixed before `σ` is in scope, so it cannot mention `σ`. -/
def Counter.unpack {ρ : Type} (c : Counter) (k : (σ : Type) → CounterOps σ → ρ) : ρ :=
  k c.State c.ops

/-- The computation rule: unpacking a package hands its contents to the client. -/
theorem Counter.unpack_pack {ρ : Type} (σ : Type) (ops : CounterOps σ)
    (k : (σ : Type) → CounterOps σ → ρ) :
    (Counter.pack σ ops).unpack k = k σ ops := rfl
```

The side condition of rule T-Unpack is enforced by scoping. The result type `ρ` is a parameter of `Counter.unpack`, fixed before the client `k` receives `σ`, so `ρ` cannot depend on `σ`. The client `k` must be a function of *every* type `σ`, which is rule T-Unpack's "fresh α". A client written against this interface works for every package, including packages with different hidden types in the same list:

```
/-- Apply `f` to `s`, `n` times. -/
def iterate {σ : Type} (f : σ → σ) : Nat → σ → σ
  | 0, s => s
  | n + 1, s => iterate f n (f s)

/-- A client: take `n` steps from the initial state and read the value. It is
written once and works for every package. -/
def Counter.run (c : Counter) (n : Nat) : Nat :=
  c.unpack (fun _ ops => ops.get (iterate ops.step n ops.init))

/-- A heterogeneous list: three packages with two different hidden types. -/
def counters : List Counter := [byOne, unary, byTen]

/-- info: [3, 3, 30] -/
#guard_msgs in
#eval counters.map (·.run 3)
```

Abstract and transparent

Inside a function that receives an arbitrary package `c`, the hidden type is `c.State`, and Lean knows nothing about it. A client that tries to treat the initial state as a number is rejected, as `peek` was in Rust:

```
/--
error: Type mismatch
  c.ops.init
has type
  c.State
but is expected to have type
  Nat
-/
#guard_msgs in
def peek (c : Counter) : Nat := c.ops.init
```

The message has the same shape as Rust's E0308 with its opaque type: `c.State` is not `Nat`. But Lean's packages are not opaque in Rust's sense. For a *specific* package, the type field is an ordinary definition, and Lean can see through it:

```
/-- For a *specific* package, Lean can see through the type field. -/
example : byOne.State = Nat := rfl

def peekOne : Nat := byOne.ops.init
```

`byOne.State` computes to `Nat`, so `byOne.ops.init` *is* a natural number. In the terminology of module systems, a dependent pair is a *transparent* package (the projections `.1` and `.2`, here `.State` and `.ops`, are always available), while System F's ∃ and Rust's `impl Trait` are *opaque*. Lean code gets abstraction from the way clients are written (a function of an arbitrary package cannot know which package it has), not from the type of the package.

The universe bump has consequences

Living in `Type 1` is not a mere technicality. Many functions and type constructors expect a type in `Type`, and a `Counter` does not qualify:

```
/--
error: Type mismatch
  Counter
has type
  Type 1
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1`
-/
#guard_msgs in
def CounterType : Type := Counter

/--
error: Application type mismatch: The argument
  Counter
has type
  Type 1
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1` in the application
  IO Counter
-/
#guard_msgs in
def chooseCounter : IO Counter := pure byOne
```

The second error is the one that matters in practice: Lean's `IO` takes a type in `Type`, so an `IO` action cannot return a package with a type field. Rust has no such restriction, because in Rust types are not values: a `Box<dyn Counter>` contains a vtable, not a type, and there is no universe of types for it to live in. In Lean the hidden type is a real component of the package. The compiler erases it at run time, but the logic has to account for it.

`Exists` versus `Sigma`, again

Chapter 10 contrasted the two ways of saying "there exists" in Lean. They meet again here:

```
/-- info: ∃ σ, Nonempty (CounterOps σ) : Prop -/
#guard_msgs in
#check ∃ σ : Type, Nonempty (CounterOps σ)

/-- The proposition is provable: `Nat`, with `natOps`, is a witness. -/
theorem some_counter_exists : ∃ σ : Type, Nonempty (CounterOps σ) :=
  ⟨Nat, ⟨natOps⟩⟩

/--
error: Invalid projection: Cannot project a value of non-propositional type
  Type
from the expression
  h
which has propositional type
  ∃ σ, Nonempty (CounterOps σ)
-/
#guard_msgs in
def witness (h : ∃ σ : Type, Nonempty (CounterOps σ)) : Type := h.1
```

`∃ σ : Type, Nonempty (CounterOps σ)` is a proposition, in `Prop`, and stays there even though it quantifies over all types: `Prop` is *impredicative* (Chapter 12), so no universe bump happens. But a proof of an `Exists` cannot be taken apart to produce data: the witness cannot be extracted, because of the elimination restriction of Chapter 10. The `Sigma`-style structure `Counter` pays the universe bump and in exchange gives full access to its contents. Rust's `dyn Counter` sits in between: like `Exists`, it does not let you extract the witness type (short of `Any`), and like `Sigma`, it is data that you can compute with.

Representation independence, proved

The promise of abstract types is that a client cannot tell two implementations apart if they behave the same through the interface. For our counters, counting with a `Nat` and counting with a `List Unit` should be indistinguishable. The file `code/lean/Book/Ch15/Independence.lean` proves this for every client written in a small command language:

```
/-- A client program: a list of commands. -/
inductive Cmd where
  | step
  | read

/-- Run a client program against an implementation, collecting the values it
reads. -/
def exec {σ : Type} (ops : CounterOps σ) : List Cmd → σ → List Nat
  | [], _ => []
  | .step :: cs, s => exec ops cs (ops.step s)
  | .read :: cs, s => ops.get s :: exec ops cs s
```

A client program is a list of commands; `exec` runs it against an implementation, starting from a given state, and collects the values that the `read` commands observe. The proof technique is a **simulation**:

a relation between the states of the two implementations that holds at the start, is preserved by `step`, and relates only states that `get` reports equally.

```
/-- A simulation between two implementations: a relation between their
states that holds initially, is preserved by `step`, and relates only states
that `get` cannot tell apart. -/
structure Simulation {σ τ : Type} (A : CounterOps σ) (B : CounterOps τ) where
  R : σ → τ → Prop
  init : R A.init B.init
  step : ∀ s t, R s t → R (A.step s) (B.step t)
  get : ∀ s t, R s t → A.get s = B.get t
```

```
/-- Related states produce the same outputs, for every program. -/
theorem exec_sim {σ τ : Type} {A : CounterOps σ} {B : CounterOps τ}
    (sim : Simulation A B) :
    ∀ (cs : List Cmd) (s : σ) (t : τ), sim.R s t → exec A cs s = exec B cs t := by
  intro cs
  induction cs with
  | nil =>
    -- No commands: both runs output the empty list.
    intro s t _
    rfl
  | cons c cs ih =>
    intro s t hR
    cases c with
    | step =>
      -- Both sides take a step; the new states are related again.
      exact ih (A.step s) (B.step t) (sim.step s t hR)
    | read =>
      -- Both sides output a value (equal, by `sim.get`) and continue
      -- from the same, related states.
      show A.get s :: exec A cs s = B.get t :: exec B cs t
      rw [sim.get s t hR, ih s t hR]

/-- Hence the two implementations are indistinguishable from their initial
states. -/
theorem indistinguishable {σ τ : Type} {A : CounterOps σ} {B : CounterOps τ}
    (sim : Simulation A B) (cs : List Cmd) :
    exec A cs A.init = exec B cs B.init :=
  exec_sim sim cs A.init B.init sim.init
```

The proof of `exec_sim` is by induction on the program. Notice that it introduces only `cs` before the induction, so that the induction hypothesis `ih` speaks about *all* pairs of related states, which the step case needs (the same pattern as `generalizing` in Chapter 14). For the empty program, both runs output `[]`. For a program that starts with `step`, both implementations take a step, the new states are related by `sim.step`, and the induction hypothesis applies to them. For a program that starts with `read`, both runs output a value and continue from the same states; `show` exposes the two lists, `sim.get` proves that the outputs are equal, and the induction hypothesis proves that the rest are equal. The corollary `indistinguishable` applies this to the initial states, which `sim.init` relates.

For the two counters, the relation is "the list's length is the number":

```
/-- The simulation between counting with `Nat` and counting with a list of
units: the list's length is the number. -/
def natUnary : Simulation natOps unaryOps where
  R n xs := xs.length = n
  init := rfl
  step n xs h := by
    -- h    : xs.length = n
    -- Goal : R (n + 1) (() :: xs), that is, (() :: xs).length = n + 1
    show xs.length + 1 = n + 1
    rw [h]
```

```
  get n xs h := by
    -- Goal : natOps.get n = unaryOps.get xs, that is, n = xs.length
    show n = xs.length
    rw [h]

/-- No client program can tell the two implementations apart. -/
theorem nat_unary (cs : List Cmd) :
    exec natOps cs natOps.init = exec unaryOps cs unaryOps.init :=
  indistinguishable natUnary cs
```

Each obligation is a one-line rewrite with the hypothesis `h`, after `show` has unfolded the operations (`(() :: xs).length` is `xs.length + 1` by definition). The theorem `nat_unary` then says that no program in the command language can tell the two representations apart. A counter that counts by ten, on the other hand, is told apart by a two-command program, and `decide` checks it:

```
/-- Counting by ten, on the other hand, is told apart by a two-command
program. -/
example : exec natOps [.step, .read] natOps.init ≠
    exec tenOps [.step, .read] tenOps.init := by
  decide
```

The command language makes "every client" precise enough to prove something. The full statement, for every client of type $\forall S.\, \mathit{CounterOps}\ S \to \rho$, is an instance of **parametricity**, which says that polymorphic functions preserve relations between types; Chapter 18 explains why it cannot be proved inside Lean, and Rust's `Any` shows that it fails when the language lets clients inspect types.

15.7 Encoding ∃ with ∀

There is a remarkable fact about existential types: you do not need them as a primitive, because they can be encoded with universal ones.

$$\exists\alpha.\,\tau \cong \forall\beta.\,(\forall\alpha.\,\tau \to \beta) \to \beta$$

Read the right side as a description of what you can *do* with a package: for any result type β, if you give me a client that works for every α (a function $\forall\alpha.\,\tau \to \beta$), I will run it and return its result. Packing is "remember σ and v, and when a client arrives, run it on them"; unpacking is "hand the package your client":

$$\mathit{pack}\ \langle\sigma, v\rangle = \Lambda\beta.\,\lambda k.\,k\,[\sigma]\,v \qquad\qquad \mathit{unpack}\ \langle\alpha, x\rangle = p\ \mathit{in}\ e = p\,[\rho]\,(\Lambda\alpha.\,\lambda x.\,e)$$

The side condition of T-Unpack is built in: the client is a polymorphic function, so it cannot know α, and its result type ρ is chosen before α is bound. In logic, the encoding is the fact that $\exists x.\,P(x)$ holds exactly when, for every conclusion Q, "for all x, $P(x)$ implies Q" implies Q. In programming, it is *continuation-passing style*: instead of returning a package, call the continuation with its contents.

In Rust

Rust cannot write the type $\forall\alpha.\,\tau \to \beta$ as a closure type, because closures are not generic (Chapter 13 showed E0308 when one closure is used at two types). Chapter 12's workaround applies: a rank-2 type is a trait with a generic method. The client is a *visitor*:

```
/// A client that works for *every* counter type: ∀C. Counter C ⇒ C → Output.
/// Closures cannot be generic, so the client is a trait with a generic method.
trait CounterClient {
    type Output;
    fn visit<C: Counter>(self, counter: C) -> Self::Output;
}

/// The existential, encoded with a universal: instead of returning a counter
/// of a hidden type, take a client and hand it the counter.
/// ∃C. Counter C ∧ C  ≅  ∀R. (∀C. Counter C ⇒ C → R) → R
fn with_counter<K: CounterClient>(kind: &str, client: K) -> K::Output {
    match kind {
        "simple" => client.visit(Simple(0)),
        "logged" => client.visit(Logged {
            count: 0,
            log: Vec::new(),
        }),
        _ => client.visit(Stride { value: 0, by: 10 }),
    }
}

/// One client: take some steps and report the value.
struct StepAndGet(u32);

impl CounterClient for StepAndGet {
    type Output = u32;
    fn visit<C: Counter>(self, counter: C) -> u32 {
        run_generic(counter, self.0)
    }
}

/// Another client, with a different result type: the name of the hidden type,
/// which this client may print but not use.
struct TypeName;

impl CounterClient for TypeName {
    type Output = &'static str;
    fn visit<C: Counter>(self, _counter: C) -> &'static str {
        std::any::type_name::<C>()
    }
}
```

`with_counter` is the package, in encoded form. It does not return a counter; it receives a client and calls `client.visit` with the counter it chooses. Each arm of the `match` calls `visit` at a *different* type, which is fine, because `visit` is generic, and the compiler monomorphizes it three times. So this encoding achieves what `impl Counter` could not (choosing the hidden type at run time) without a `Box` or a vtable:

```
for kind in ["simple", "logged", "stride"] {
    let value = with_counter(kind, StepAndGet(3));
    let name = with_counter(kind, TypeName);
    println!("cps:        {kind:<7} -> {value:>2}  (hidden type {name})");
}
```

```
cps:        simple  ->  3  (hidden type ch15_existentials::Simple)
cps:        logged  ->  3  (hidden type ch15_existentials::Logged)
cps:        stride  -> 30  (hidden type ch15_existentials::Stride)
```

The two clients have different result types (`u32` and `&'static str`): that is the $\forall\beta$ of the encoding, here the associated type `Output`. The second client, `TypeName`, shows the parametricity leak mentioned above: `type_name` prints the hidden type, which the client was not supposed to know. The price of the encoding is inversion of control: every client must be written as a type that implements a trait,

rather than as a closure. The `serde` library's `Visitor` traits are a well-known use of this pattern. Note also that the encoding could not be turned into a trait object: a trait with a generic method is not dyn compatible.

In Lean

In Lean, the encoding is a definition, and its properties are theorems (`code/lean/Book/Ch15/Encoding.lean`):

```
/-- ∃α. τ α, encoded: for every result type `ρ`, given a client that works
for every `α`, produce a `ρ`. The result types range over `Type u`. -/
def Hidden (τ : Type → Type) : Type (u + 1) :=
  (ρ : Type u) → ((α : Type) → τ α → ρ) → ρ

/-- Pack: remember `α` and `x`; when a client arrives, run it on them. -/
def Hidden.pack {τ : Type → Type} (α : Type) (x : τ α) : Hidden.{u} τ :=
  fun _ k => k α x

/-- Unpack: hand the client to the package. -/
def Hidden.unpack {τ : Type → Type} {ρ : Type u} (p : Hidden.{u} τ)
    (k : (α : Type) → τ α → ρ) : ρ :=
  p ρ k

/-- The computation rule holds by definition. -/
theorem Hidden.unpack_pack {τ : Type → Type} {ρ : Type u} (α : Type) (x : τ α)
    (k : (α : Type) → τ α → ρ) :
    (Hidden.pack α x).unpack k = k α x := rfl
```

`Hidden τ` is $\forall \rho.\, (\forall \alpha.\, \tau\, \alpha \to \rho) \to \rho$, with the result types ρ ranging over the universe `Type u`. `Hidden.pack` and `Hidden.unpack` are the two definitions above, and the computation rule of existentials, unpacking a package hands its contents to the client, holds by `rfl`: it is literally β-reduction. Here is Rust's `with_counter`, with the counter operations of `Existential.lean`:

```
/-- The Rust function `with_counter`, in Lean: the package is chosen at run
time, and each branch packs a different state type. -/
def withCounter (fast : Bool) : Hidden.{0} CounterOps :=
  if fast then Hidden.pack Nat tenOps
  else Hidden.pack (List Unit) unaryOps

/-- A client: three steps, then read. It must work for every state type. -/
def stepThree (σ : Type) (ops : CounterOps σ) : Nat :=
  ops.get (ops.step (ops.step (ops.step ops.init)))

/-- info: (30, 3) -/
#guard_msgs in
#eval ((withCounter true).unpack stepThree, (withCounter false).unpack stepThree)
```

Universes appear again. The encoding quantifies over all result types in `Type u`, so it lives one universe higher:

```
/-- info: Hidden CounterOps : Type 1 -/
#guard_msgs in
#check Hidden.{0} CounterOps

/-- info: Hidden CounterOps : Type 2 -/
#guard_msgs in
#check Hidden.{1} CounterOps
```

With ρ ranging over `Type` (universe level 0), `Hidden CounterOps` is in `Type 1`, the same universe as the dependent pair. Can we convert between the two? One direction is easy, but the other needs the

client to return a *dependent pair*, a type in `Type 1`, so the result types must range over `Type 1`, and the encoding moves up to `Type 2`:

```
/-- From a dependent pair to the encoding. The result types must range over
`Type 1`, because the pair type itself lives in `Type 1`. -/
def toHidden {τ : Type → Type} : ((α : Type) × τ α) → Hidden.{1} τ
  | ⟨α, x⟩ => Hidden.pack α x

/-- Back: use the pair type itself as the result type `ρ`. -/
def fromHidden {τ : Type → Type} (p : Hidden.{1} τ) : (α : Type) × τ α :=
  p.unpack (fun α x => ⟨α, x⟩)

/-- One round trip is the identity, by computation. -/
theorem fromHidden_toHidden {τ : Type → Type} (s : (α : Type) × τ α) :
    fromHidden (toHidden s) = s := by
  -- Split the pair into its type and its value; then both sides compute
  -- to the same pair.
  cases s
  rfl
```

Going from a pair to the encoding and back gives the pair you started with, and the proof is a computation: after `cases` splits the pair into its type and its value, both sides reduce to the same pair. The other round trip, from an encoded package `p` to a pair and back, should also give `p` back, but proving it requires knowing that `p` treats its client's type parameter uniformly. That is a free theorem, true of every `p` you can write without inspecting types, and, as Chapter 18 explains, not provable inside Lean. In System F, whose polymorphism is impredicative, the $\forall\beta$ ranges over all types, including the encoding itself, so everything stays in one universe (and, with parametricity, the two sides are isomorphic); in a predicative hierarchy like Lean's, each step up costs a universe.

In `Prop`, which is impredicative, none of these problems arise, and the encoding is a theorem in both directions:

```
/-- In `Prop`, the encoding is an equivalence, and both directions are
provable: a proof of `∃ x, P x` is exactly something that can serve every
client `∀ x, P x → Q`. -/
theorem exists_iff_forall {α : Type} (P : α → Prop) :
    (∃ x, P x) ↔ ∀ Q : Prop, (∀ x, P x → Q) → Q := by
  constructor
  · -- Unpack the witness and hand it to the client `k`.
    intro ⟨x, hx⟩ Q k
    exact k x hx
  · -- Choose the client that packs its argument again.
    intro h
    exact h (∃ x, P x) (fun x hx => ⟨x, hx⟩)

/-- info: 'Ch15.exists_iff_forall' does not depend on any axioms -/
#guard_msgs in
#print axioms exists_iff_forall
```

From left to right: take the witness `x` and its proof `hx` apart, and hand them to the client `k`. From right to left: the hypothesis `h` works for every conclusion `Q`, so choose `Q := ∃ x, P x` and the client that packs its arguments again. The proof uses no axioms at all. Since proofs of propositions carry no information beyond their existence, the round trip is automatic.

15.8 Putting it together: choosing where the quantifier goes

Let us use the chapter's ideas on a concrete design question. A lab control program reads a configuration string that names a counter implementation (`"simple"`, `"logged"` or `"stride"`), and must run the chosen counter. Which Rust feature should express "the counter chosen by the configuration"?

A generic parameter does not fit. `fn run<C: Counter>(c: C)` lets the *caller* choose `C`, at compile time. Here the choice is made by a string, at run time, inside the function that reads the configuration.

`impl Trait` in return position does not fit either. A function `fn from_config(name: &str) -> impl Counter` must return one type, decided at compile time, and the compiler rejects two arms that return different types (E0308, as we saw). The existential is there, but its witness is fixed per function.

`dyn Trait` fits. `make_counter_dyn` returns a `Box<dyn Counter>`: each call may choose a different type, each value carries its vtable, and any client of `&mut dyn Counter` can use it. The costs are a heap allocation, an indirect call per method, and the restriction that `Counter` must be dyn compatible. That is the usual answer, and the compiler's own suggestion.

The encoding fits too. `with_counter` takes a `CounterClient` and calls it with the chosen counter: three monomorphized calls, no allocation, no vtable, and no dyn-compatibility requirement. The price is that the code that uses the counter must be written as a client type with a generic method, and that the counter cannot outlive the call.

And if the counters must be combined, as in `merge`, neither objects nor the encoding are enough: binary operations need several values with *the same* hidden type, which is the module-style existential, a trait with an associated type such as `CounterModule`.

Whichever you choose, the Lean side tells you what is guaranteed. A client that uses only the interface cannot distinguish `Simple` from `Logged` (in Lean, `natOps` from `unaryOps`), because there is a simulation between them, and `nat_unary` proves that no program in the command language observes a difference. A client that uses `Any`, `type_name` or `size_of` can, and the `TypeName` client did: abstraction holds only as long as clients play by the rules of T-Unpack.

15.9 Common confusions

"**`impl Trait` always means an existential.**" Only in return position. In argument position, `fn f(x: impl Trait)` is a generic function: the caller chooses the type. The same syntax denotes $\forall$ in one position and $\exists$ in the other.

"**`impl Trait` hides the type from the compiler.**" It hides it from the *caller's code*. The compiler knows it, generates direct calls, and even lets auto traits such as `Send` leak through. Only `dyn Trait` erases the type at run time.

"**`dyn Trait` is a type like any other.**" It is a dynamically sized type: it has no size known at compile time, so you can only use it behind a pointer (`&dyn Trait`, `Box<dyn Trait>`, `Rc<dyn Trait>`), and that pointer is two words long.

"**A trait that is not dyn compatible is badly designed.**" It may simply describe a module-style abstraction rather than objects. `Clone`, `Default`, `PartialEq` with its binary `eq`, and Chapter 14's `Monoid` are all excellent traits that are not dyn compatible, because their methods return `Self`, lack a receiver, or take a second `Self`.

"**A Lean structure with a type field hides that type.**" It does so only from code that receives an arbitrary package. For a specific package, the field computes to the actual type. Lean's dependent pairs are transparent; Rust's opaque types are not.

15.10 Summary

- The existential type $\exists\alpha.\,\tau$ packages a hidden type with a value. Rule T-Pack forgets the type; rule T-Unpack lets a client use the package for an arbitrary α, with the side condition $\alpha \notin \mathrm{ftv}(\rho)$ (the eigenvariable condition of ∃E) keeping the type from escaping. That side condition is what makes types abstract.
- Abstract data types have existential type (Mitchell and Plotkin, 1988), and representation independence is the guarantee that clients cannot distinguish implementations related by a simulation; we proved it in Lean for two counters and every program of a command language.
- Rust offers three placements. Generics and `impl Trait` in argument position are ∀ (the caller chooses). `impl Trait` in return position is ∃ with one statically known, opaque witness per function. `dyn Trait` is ∃ with a witness per value, chosen at run time. A trait with an associated type gives module-style existentials.
- A pointer to a `dyn Trait` is fat: a data pointer and a vtable pointer (two words, as `size_of` shows). The vtable is the dictionary of Chapter 14 plus the drop function, size and alignment of the erased type. Slices are fat pointers too, hiding a length.
- A trait is dyn compatible only if every method can be called through a single vtable entry on one unpacked value: no `Self` in the result or in other arguments, no type parameters, a receiver. `where Self: Sized` and trait-object parameters are the standard repairs (E0038).
- `dyn Any` and downcasting reveal the hidden type at run time. They are useful (type maps, error inspection) and they break abstraction and parametricity (Chapter 18), as do `size_of` and `type_name`.
- $\exists\alpha.\,\tau \cong \forall\beta.\,(\forall\alpha.\,\tau \to \beta) \to \beta$: in Rust, a client trait with a generic method gives run-time choice of type without a vtable; in Lean, the encoding computes, one round trip is `rfl`, the other is a free theorem, and each step up in the encoding costs a universe. In `Prop` the encoding is a theorem in both directions.
- In Lean, packages are dependent pairs such as `(α : Type) × α × (α → String)` or structures with a type field. They live in `Type 1`, which has practical consequences (`IO Counter` is ill-typed). `Exists` stays in `Prop` but cannot release its witness.

15.11 Exercises

1. For each signature, say whether the type is universally or existentially quantified, who chooses it, and when it is known: `fn a(x: impl Iterator<Item = u8>)`, `fn b() -> impl Iterator<Item = u8>`, `fn c() -> Box<dyn Iterator<Item = u8>>`, `fn d<I: Iterator<Item = u8>>(x: I)`. *Hint:* two are universal and identical in meaning; the two existentials differ in when the hidden type is fixed.
2. Explain, using rule T-Unpack, why a method `fn duplicate(&self) -> Self` makes a trait not dyn compatible, and why adding `where Self: Sized` to that method makes the trait dyn compatible again. Can `duplicate` then be called on a `Box<dyn Trait>`? *Hint:* look at the result type of the unpacked call; `dyn Trait` is never `Sized`, so the method is not in the vtable.
3. In Rust, write the CPS encoding of `Box<dyn Display>`: a trait `ShowClient { type Output; fn visit<T: Display>(self, value: T) -> Self::Output; }` and a function `with_reading(kind: &str, client: K) -> K::Output` that chooses between an `i32`, a `String` and an `f64`. Write a client that formats the value inside brackets. *Hint:* follow `with_counter` and `StepAndGet`; the client's `visit` can use `format!("[{value}]")` because of the `Display` bound.
4. In Lean, define `twoOps : CounterOps Nat` whose state counts by two and whose `get` divides by two, and prove that it is indistinguishable from `natOps` by constructing a `Simulation natOps twoOps`. *Hint:* take `R n m := m = 2 * n`. Each obligation becomes a goal about `Nat` after `show`; `omega` proves all of them, including the one with division.
5. In Lean, use `nat_unary` to prove that `byOne.run n = unary.run n` for every `n`. *Hint:* first prove a lemma relating `exec` on the program `List.replicate n .step ++ [.read]` to `iterate`, by induction on `n` generalizing the starting state; then both sides of the goal are the single element of an output list.
6. Pen and paper. Why does `∃ σ : Type, Nonempty (CounterOps σ)` live in `Prop` while `(σ : Type) × CounterOps σ` lives in `Type 1`? What would go wrong if `Sigma` over `Type` were allowed to live in `Type`? *Hint:* recall from Chapter 12 which of Lean's universes is impredicative, and the remark on Girard's paradox in this chapter.